

Бележки С#

```
public static DataTable GetTable()    // декларираме таблицата и колоните в нея
{
```

```
    DataTable table = new DataTable();
```

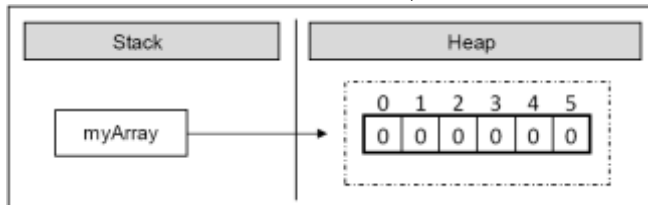
```
    DataColumn myDataColumn = new DataColumn();
```

[DataColumn\(String\)](#) Initializes a new instance of the DataColumn class, as type string, using the specified column name.

В С# масив се създава с ключовата дума **new**, която служи за заделяне (алокиране) на памет:

```
int[] myArray = new int[6];
```

В примера заделяме масив с размер 6 елемента от целочисления тип **int**. Това означава, че в динамичната памет (heap) се заделя участък от 6 последователни цели числа, които се инициализират със стойност 0:



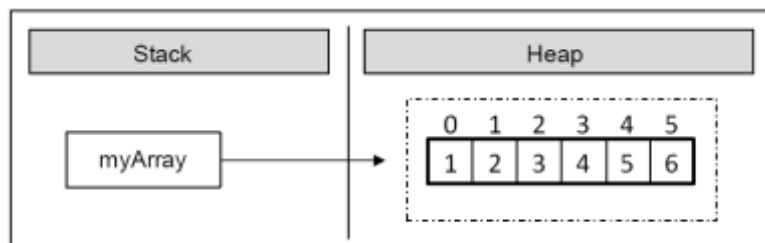
Картинката показва, че след заделянето на масива променливата **myArray** сочи към адрес в динамичната памет, където се намира нейната стойност. Елементите на масивите винаги се съхраняват в динамичната памет (в т. нар. **heap**).

При заделянето на масив в квадратните скоби се задава броят на елементите му (цяло неотрицателно число) и така се фиксира неговата дължина. Типът на елементите се пише след **new**, за да се укаже за какви точно елементи трябва да се задели памет

```
DateTime d = new DateTime(2009, 10, 23, 15, 30, 22);
```

```
int[] myArray = { 1, 2, 3, 4, 5, 6 };
```

В този случай създаваме и инициализираме елементите на масива едно-временно. Ето как изглежда масивът в паметта, след като стойностите му са инициализирани още в момента на деклариране:



```
myDataColumn.DataType = System.Type.GetType("System.Char");
```

```
myDataColumn.ColumnName = "Symbol";
```

```
myDataColumn.ReadOnly = false;
```

```
table.Columns.Add(myDataColumn);
```

```
myDataColumn = new DataColumn();
```

```
myDataColumn.DataType = System.Type.GetType("System.Int32");
```

```
myDataColumn.ColumnName = "Frequency";
```

```

myDataColumn.ReadOnly = false;
table.Columns.Add(myDataColumn);
myDataColumn = new DataColumn();
myDataColumn.DataType = System.Type.GetType("System.String");
myDataColumn.ColumnName = "Code";
myDataColumn.ReadOnly = false;
table.Columns.Add(myDataColumn);
return table;
}

```

```

// функцията, която се изпълнява при натискане на бутон "Encryption"
private void Coding_Button(object sender, EventArgs e) {
    char[] letter = new char[EncryptionTextBox.Text.Length];
    int[] value = new int[EncryptionTextBox.Text.Length];
    string sToCoding = EncryptionTextBox.Text.ToString();
    bool exist = false;
    int i = 0;
    int lenght = 0;

    for (int j = 0; j < sToCoding.Length; j++) // обхожда, докато не свърши
    съобщението
    {
        while (letter[i] != '\0') // докато стрингът не е празен
        {
            if (sToCoding[j] == letter[i]) // ако има повторение
            {
                exist = true;
                break;
            }
            i++;
        }
        if (exist) // ако съществува повторение, прибавяме към i
        {
            value[i]++;
        }
        else // добавяме буквата към string
        {
            letter[i] = sToCoding[j];
            value[i] = 1;
            lenght++;
        }
        exist = false;
        i = 0;
    }
    Array.Sort(value, letter); // сортираме в намаляващ ред (по големина на
    честотата)
    Array.Reverse(value);
    Array.Reverse(letter);
    string[] code = new string[lenght];
}

```

```

code = CodeGenration(value, code, lenght);
CodedTextBox.Text = Output(sToCoding, letter, code);
DataTable dt = DataTableGeneration(letter, value, code);
dataGridView1.DataSource = Form1.DataTableGeneration(letter, value, code);
}

```

```

public static DataTable DataTableGeneration(char[] letter, int[] value, string[] code)
{
    DataTable dt = new DataTable();
    dt = GetTable();
    for (int i = 0; i < code.Length; i++) // попълваме редовете на таблицата
    {
        DataRow dr = dt.NewRow();
        dr[0] = letter[i];
        dr[1] = value[i];
        dr[2] = code[i];
        dt.Rows.Add(dr);
    }
    return dt;
}

```

// на всяко прехвърляне търсим средата

```
private string[] CodeGenration(int[] value, string[] code, int lenght)
```

```

{
    int iHalf = FindTheHalf(value, lenght);
    int i = value[0];
    int j = 0;
    while (i <= iHalf) // преди да достигнем средата
    {
        code[j] += "0"; // присвояваме "0"
        i += value[j + 1];
        j++;
    }
    if (j > 1) // на всички думи в първата половина
    {
        int[] var = new int[j];
        string[] cd = new string[j];
        for (int y = 0; y < j; y++)
        {
            var[y] = value[y];
            cd[y] = code[y];
        }
        cd = CodeGenration(var, cd, j);
        for (int z = 0; z < j; z++)
        {
            code[z] = cd[z];
        }
    }
    for (int a = j; a < lenght; a++) // на всички останали "1"
    {

```

```

        code[a] += "1";
    }
    if (lenght - j > 1)
    {
        int[] var = new int[lenght - j];
        string[] cd = new string[lenght - j];
        for (int b = j; b < lenght; b++)
        {
            var[b - j] = value[b];
            cd[b - j] = code[b];
        }
        cd = CodeGenration(var, cd, lenght - j);
        for (int c = j; c < lenght; c++)
        {
            code[c] = cd[c - j];
        }
    }
    return code;
}

```

`private int FindTheHalf(int[] value, int lenght) // определя половината на всяка стъпка`

```

{
    double sum = 0;
    int before = 0;
    int after = 0;
    int half = 0;
    for (int i = 0; i < lenght; i++) // докато достигнем цялата дължина
    {
        sum += value[i]; // събираме вероятностите на символите
    }
    for (int i = 0; i < lenght; i++)
    {
        before = after;
        after += value[i];
        if (after >= sum / 2) // ако втората половина на сборовете е по-голяма
        {
            if ((after - sum / 2) >= (sum / 2 - before)) // ако after е по-"далече" до 50% от
before (втората половина е с по-голяма разлика в сума до средата на първата)
            {
                half = before; // вземемe parvata
            }
            else
            {
                half = after; // ina4e vtorata
            }
            break;
        }
    }
    return half;
}

```

```

    }

// изкарва кодовите думи
private string Output(string sToCoding, char[] letter, string[] code)
{
    string outputText = "";

    for (int i = 0; i < sToCoding.Length; i++)
    {
        for (int j = 0; j < code.Length; j++)
        {
            if (sToCoding[i] == letter[j])
            {
                outputText += code[j];
                break;
            }
        }
    }

    return outputText;
}

private void Clear_Button(object sender, EventArgs e) // изчиства данните в 3-те
форми
{
    EncryptionTextBox.Text = "";
    CodedTextBox.Text = "";
    dataGridView1.DataSource = Form1.GetTable();
}

```

Масивите могат да се обхождат с помощта на някоя от конструкциите за **цикъл**, като най-често използван е класическият **for** цикъл:

```

int[] arr = new int[5];
for (int i = 0; i < arr.Length;
i++)
{
arr[i] = i;
}

```

В следващия пример ще видим как може да променяме елементите на даден масив като ги достъпваме по индекс. Целта на задачата е да се подредят в обратен ред (отзад напред) елементите на даден масив. Ще обърнем елементите на масива, като използваме помощен масив, в който да запазим елементите на първия, но в обратен ред. Забележете, че дължината на масивите е еднаква и остава непроменена след първоначалното им заделяне

ArrayReverseExample.cs

```

class ArrayReverseExample
{
static void Main()
{
int[] array = { 1, 2, 3, 4, 5 };
// Get array size
int length = array.Length;
// Declare and create the reversed array
int[] reversed = new int[length];
// Initialize the reversed array
for (int index = 0; index < length; index++)
{
reversed[length - index - 1] = array[index];
}
// Print the reversed array
for (int index = 0; index < length; index++)

{
Console.Write(reversed[index] + "
");
}
}
}
// Output: 5 4 3 2 1

```

Примерът работи по следния начин: първоначално създаваме едномерен масив от тип `int` и го инициализираме с цифрите от 1 до 5. След това запазваме дължината на масива в целочислената променлива `length`. Забележете, че се използва свойството `Length`, което връща броя на елементите на масива. В C# всеки масив знае своята дължина.

След това декларираме масив `reversed` със същия размер `length`, в който ще си пазим елементите на оригиналния масив, но в обратен ред.

За да извършим обръщането на елементите използваме цикъл `for`, като на всяка итерация увеличаваме водещата променлива `index` с единица и така си осигуряваме последователен достъп до всеки елемент на масива `array`. Критерият за край на цикъла ни подсигурява, че масивът ще бъде обходен от край до край.

Нека проследим последователно какво се случва при итериране върху масива `array`. При първата итерация на цикъла, `index` има стойност 0. С `array[index]` достъпваме първия елемент на масива `array`, а съответно с `reversed[length - index - 1]` достъпваме последния елемент на новия масив `reversed` и извършваме присвояване. Така на последния елемент на `reversed` присвоихме първия елемент на `array`. На всяка следваща итерация `index` се увеличава с единица, позицията в `array` се увеличава с единица, а в `reversed` се намаля с единица.

В резултат обърнахме масива в обратен ред и го отпечатахме. В примера показвахме последователно обхождане на масив, което може да се извърши и с другите видове цикли (например `while` и `foreach`).